Designing and Conducting More Equitable Code Interviews

Arpan Kapoor*
University of Washington
Seattle, WA, USA
akap1204@uw.edu

Suhas Kannam*
University of Washington
Seattle, WA, USA
ksuhas16@uw.edu

Asmi Sathaye*
University of Washington
Seattle, WA, USA
sathaye@uw.edu

Suh Young Choi*
University of Washington
Seattle, WA, USA
atobdura@uw.edu

Gina Tran*
University of Washington
Seattle, WA, USA
gtran@uw.edu

Zoe Huang*
University of Washington
Seattle, WA, USA
zoeh1107@uw.edu

Aileen Kuang
University of Washington
Seattle, WA, USA
aileenk@uw.edu

Kevin Lin
University of Washington
Seattle, WA, USA
kevinl@cs.uw.edu

ABSTRACT

Code interviews are a promising oral assessment of student learning outcomes on take-home programming assignments. Further, code interviews can be beneficial to students entering the workforce, as it helps them practice technical communication skills in a structured setting. However, prior work has indicated that code interviews may can increase stress and exacerbate assessment inequities for students with diverse abilities and fluency with verbal communication. To mitigate these effects, we have incorporated elements of spaced learning, reduced testing frequency, and repetition learning to code interviews so that they became less stressful, more equitable and accessible, and more accurately reflected student learning. We introduced post-lecture exit tickets and guided peer activities during section designed to strengthen student understand of learning objectives and prepare students for the communication expectations during interviews. During the interview session, students were given time to read the questions, plan their answers, and prepare for technical communication before completing individual interviews with a TA. Together, we have found that these elements support a more equitable interview experience. This experience report describes the principles and structure of the redesigned code interviews and provides recommendations for others who may adopt similar oral assessments.

CCS CONCEPTS

• **Social and professional topics → Computing education**.

KEYWORDS

code interviews; oral assessment; introductory programming; accessibility; data programming; data science

*Authors contributed equally to this work

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Arpan Kapoor, Suhas Kannam, Asmi Sathaye, Suh Young Choi, Gina Tran, Zoe Huang, Aileen Kuang, and Kevin Lin. 2018. Designing and Conducting More Equitable Code Interviews. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Unlike traditional written exams, oral assessments provide the opportunity to gauge not only a student's understanding of the material but also their ability to articulate and explain their ideas in real time. Oral assessments allow instructors to observe students' communication skills, problem-solving processes, and critical thinking abilities in a more interactive and authentic setting than traditional written exams [6]. Studies have shown that oral assessments can foster critical thinking, adaptability, and improved verbal communication skills, which are crucial skills for academic and professional success[5]. Oral exams can be a more comprehensive and equitable method for assessing student learning, particularly in contexts that emphasize problem-solving and technical communication [8]. In the context of introductory programming education, oral assessments have recently taken the form of **code interviews** wherein students solve questions posed live by an instructor [1, 3].

As a form of authentic assessment, code interview draws some of its legitimacy from practices in the tech industry. By improving our practices for code interview in the classroom, we also may better prepare students for interviews and technical communication in the real-world. For example, when debugging code during an interview, students might describe why a feature doesn't work properly, explain the logic behind their fix, and defend their approach. This mirrors situations in a job interview or team meeting where they will need to clearly and logically guide others through their work, ensuring their reasoning is sound and easy to understand [2]. It also prepares them for the real world, where they may need to break down technical concepts into simpler terms for non-technical colleagues, stakeholders, or managers to understand [4]. For example, during a cross-functional meeting or presentation, explaining how a data pipeline processes information or why a particular algorithm

works may be crucial. By practicing these skills regularly in a structured but low-pressure environment, students can develop their code reasoning skills, communicate their ideas clearly, and identify the key elements of program logic that contribute to high-quality code.

However, while oral assessments like code interviews offer potential benefits, they can increase stress and anxiety, particularly for minoritized students, students who are unfamiliar with the format, and students with disabilities or English language learners [3, 7]. In this experience report, we propose a more equitable approach to code interviews that is designed to reduce stress and increase accessibility for a wider range of students. Specifically, we present a code interview protocol that offers students a way to prepare for code interviews in the form of lightweight pre-interview exit tickets, quiet time for students to prepare their live responses immediately before their interview, and incorporating peer-learning opportunities and practice interviews prior to the first graded interview with a TA. These changes not only aim to alleviate stress but also create a more supportive environment for students to practice and develop their professional communication skills.

2 RELATED WORK

Kannam et al. experimented several different formats for conducting code interviews ranging from highly unstructured group reflections on take-home programming work to highly structured individual programming problem solving. Through action research, they identified that unstructured group reflections were not effective for assessing student proficiency on key course learning outcomes [3]. Individual programming problem solving tasks allowed for more fine-grained evaluation of course learning outcomes by requiring students to carefully articulate their reasoning and problem solving process for the specific question. Across all formats, code interviews provided a promising way to assess students' programming skills, particularly for courses that emphasize take-home programming work.

However, Kannam et al. observed that code interviews introduced new logistical challenges. Conducting individual code interviews for over 25 students within a 50-minute section was difficult, often requiring at least three teaching assistants (TAs) to ensure timely completion. While these interviews provided the most precise feedback and assessment, they were resource intensive and not easily scalable [3]. Additionally, code interviews raised new questions around fairness between particular evaluators. While fairness in manually-graded work is always a challenge when there are multiple evaluators, the unique oral communication element to code interviews presented new difficulties.

Most importantly, code interviews represented as a very different form of assessment for students that could be stressful for students. Kannam et al. held weekly graded code interviews, matching the cadence of their weekly take-home programming assessments. For students with less prior experience explaining their code, code interviews can be discouraging if they perceive their performance as more a consequence of the unfamiliar format than their programming proficiency. Furthermore, replacing familiar programming practice with unfamiliar code interviews led to fewer opportunities for students to practice and develop their programming skills. The expectation to consistently perform within this structure may have contributed to the common theme of student stress [3].

3 MORE EQUITABLE CODE INTERVIEWS

To address these challenges, we implemented 4 major changes to equip students with the necessary skills, familiarity, and confidence for code interviews:

- In order to make space for the following 3 changes, code interviews were focused on only 2 key assignments (section 3.1).
- Weeks before the code interview, students engaged in collaborative peer discussions to practice oral communication skills with their peers (section 3.2).
- In the days preceding the code interview, students completed a lecture exit ticket to get them thinking about key concepts that will be tested during the code interview (section 3.3).
- On the day of their code interview, students received 10 minutes of quiet time to think about how they would answer the presented code interview questions (section 3.4).

By incorporating these elements, we sought to reduce stress and provide students with support to succeed in both the course and future technical interviews. This gradual exposure to content and skills, from weeks before the code interview to the day-of, aimed to reduce the cognitive load of the code interview activity, improving students' ability to focus on demonstrating their programming proficiency rather than adapting to the expectations of the format.

3.1 Focusing code interviews on the most important assignments

Unlike Kannam et al. and their structure of conducting code interviews every week for a total of 5 code interviews, we held 2 code interviews in total corresponding to the 2 most important assignments in the course. This enabled us to reorganize the remaining weeks to provide students additional practice on either (1) course concepts, addressing the practice gap that students identified [3], or (2) communication skills, addressing the stress of the unfamiliar oral assessment format.

The 2 most important assignments were selected by the course instructor as assignments that best incorporated the overall course learning outcomes. To simplify discussion, we explain our approach by describing just the first code interview that occurred during week 5 of the course corresponding to the third assignment about *Education Attainment*. Topics and skills covered in this assignment include Python programming fundamentals, the Pandas library for parsing and manipulating, data frames, and the Seaborn library for creating data frame visualizations.

3.2 Practicing oral communication with collaborative peer discussions

In the weeks leading up to the code interview, we facilitated structured group activities during TA-led sections to reinforce key concepts. These sessions began with a review of recent lecture material followed by a guided notebook exercise. Students typically started with an individual warm-up activity before transitioning into small group discussions, during which they put their laptops away to

focus on problem solving collaboratively. After sharing their ideas, they tested and refined their solutions before reviewing the answers together as a class. This process was designed to encourage peer learning.

For example, in week 4, the week leading up to the code interview, students worked on a Python notebook in section using a Mario Kart scoreboard dataset to practice essential Pandas functions like `.loc[]` and `.groupby()`, as well as visualizing the data. During the group activity, students completed tasks such as:

- Find the `Players` with the most `Coins` in each `Drivetrain`.
- Count how many `Players` in each `Playstyle` category like each `FavoriteTrack`.
- Create both a line plot and a scatter plot to visualize `TopSpeed` trends by `Playstyle`. Compare the effectiveness of each in identifying patterns or outliers.
- What if we wanted to set the index of our data frame to be 2 columns? Set the indices to `Drivetrain` and `FavoriteTrack` and then find all the `Bikers` who like `Bowser's Castle`.

3.3 Priming content knowledge with pre-interview exit tickets

Next, we introduced weekly exit tickets designed to synthesize recent lecture and assignment content. These exit tickets are released after lecture on the day before the corresponding TA-led section for that week. They contain questions that encourage students to reflect on key takeaways and apply their knowledge in a more conceptual framework. These are due by the end of the same day, so students are expected to complete them before section.

For example, the exit ticket that preceded the code interview contained 3 questions aimed at helping students reflect on their conceptual understanding of key topics. By engaging with these questions, students are encouraged to connect practical coding skills with the underlying concepts that drive their use. The questions we asked in the exit ticket are listed below:

- What parameters does `.loc[]` take, and how does it work with a multi-indexed data frame? Additionally, explain how slicing works with `.loc[]` in a multi-indexed data frame.
- When selecting a column from a data frame, what is the difference between `df['column']` and `df[['column']]`?
- When creating a plot, what key pieces of information must be provided for the plot to render correctly? Specifically, how do the `x` and `y` parameters differ from the `hue` parameter in terms of how data is visually represented?

3.4 Restructuring code interviews to reduce stress

Compared to prior work [3], code interviews were also restructured to address all previous preparatory steps with changes primarily happening in three ways: adding 10 minutes of thinking time prior to conducting an interview, and allowing students to swap a question for a similar alternative if they did not feel confident about their answer. The questions were drawn directly from the weekly take-home programming assignment and aligned with content knowledge suggested in the exit ticket.

3.4.1 10-minute thinking time with access to the question bank. Students were given 10 minutes during which a question bank consisting of two debugging and two isomorphic questions was displayed on the room's projector screen. The debugging questions provide a buggy portion of code with a corresponding error message where the student must identify the bug and explain a fix. The isomorphic question provides fully-functional code where the student must identify specific modifications needed to satisfy an alternative prompt. The 10 minutes served as thinking time for students to prepare their solutions and think about how to communicate their thoughts, after which the questions were removed from the screen. During this quiet time, students were allowed to take notes on a sheet of paper and use their notes to support their live interview response.

3.4.2 2 randomly-assigned questions with the option to swap 1. Students then began their individual code interviews by responding to 2 questions posed by their TA, one from each category. The 2 questions were selected by the TA from the question bank that students previously studied. To further reduce stress, students were given the option to swap 1 of the 2 randomly-assigned questions for a different question if they did not feel confident about their response. Students had around 5 minutes to complete their code interview, providing solutions for each question that were ultimately graded both on accuracy and quality of communication.

3.4.3 Conducting code interviews after students received feedback. Finally, the timing of code interviews in the weekly schedule was adjusted so that code interviews occurred after they received feedback from their TA on the corresponding weekly take-home programming assignment. Compared to prior work [3] that scheduled code interviews on the day the programming assignment was due, we hoped this change would give students more time to prepare and integrate feedback left by their TA on the programming assignment.

For example, in the code interview on *Education Attainment*, one of the isomorphic questions we asked prompted students to change a fully-functional implementation that bar-plotted the percentage of people with at least high school completion in a given year (colored by sex) to instead compare the total percentages of all educational attainment levels with sex `F` in the given year (coloring optional). To successfully address this problem, students needed to make 3 changes:

(1) Change the visualization selection from `(year, slice(None), 'high school')` to `(year, 'F', slice(None))`.
(2) Change the parameter `x='Sex'` to `x='Min degree'` in order to compare educational attainment levels across the x-axis of the bar plot.
(3) Either change the parameter `hue='Sex'` to `hue='Min degree'` to color each bar a different color, or remove the `hue` parameter entirely so that all the bars appear with the same color.

One of the debugging questions we asked was designed to evaluate students' understanding of hierarchical indexing, which was a skill taught in class and practiced over each task in the assignment (such as in change 1 for the isomorphic question example above). The bug involved forgetting to include the `year` value for the visualization selection, which resulted in a `KeyError` as the data frame

would attempt to match the sex value (such as the string F) against a column of integer years.

4 REFLECTION & DISCUSSION

In this section, we report on preliminary findings from the teaching team over 2 offerings that used the more equitable code interview protocol. The teaching team, which includes the instructor and all TAs, met weekly during each course offering to discuss the past week's events and plan ahead for the following week. TAs responsible for conducting the code interviews offered both positive and mixed reflections on their effectiveness. These reflections have been organized into the following themes.

4.1 Focusing on 2 code interviews helped balance assessment and learning, but still required additional staffing to conduct interviews

Instead of code interviews taking place every week, the revised cadence focusing only on 2 code interviews (section 3.1) helped address concerns raised in prior work about course culture over-emphasizing assessment at the cost of learning [3].

Although conducting fewer code interviews is almost certainly less work than conducting more code interviews, the revised cadence still presented an additional workload demand on TAs to scale one-on-one interviews with each student in the class. Interviewing every student in a 30-student section within a 50-minute time period (with 10 minutes deducted for thinking time) required triple- or quadruple-staffing each section during interview weeks to maintain a sufficient TA-to-student ratio. However, considering that code interviews may be used in place of traditional written assessments, the time that would have been spent grading pencil-and-paper exams could instead be budgeted toward conducting code interviews.

4.2 Exit tickets and thinking time provided useful scaffolding but may have also interfered with the accuracy of the assessment grades

The teaching team reported that exit tickets helped a lot (section 3.3). By preparing students in advance not only for general topics but also for specific content knowledge they would need during the code interview, TAs observed that students were often prepared to make progress on each problem. Thinking time (section 3.4.1) further enabled students to apply their prior proficiency toward the new questions presented to them.

We also offered students flexibility to swap-out a question that was assigned to them for another question of the same type (section 3.4.2). Despite including the specific content knowledge necessary for every question in the exit tickets, there were still some students who chose to skip questions, and these seemed to be more common for certain content knowledge. While the exact frequencies and patterns will be specific to each course conducting code interviews in this fashion, this could indicate broader variability in student comfort level across different course content knowledge.

Furthermore, by allowing students to view the entire 4-question bank during thinking time, some students were able to make progress in ways that surprised the teaching team. For instance, in one case, a student appeared to work toward an answer to one question by noticing a pattern in the examples in the syntax that appeared in other questions. While this creative problem solving approach is certainly commendable on its own right, this skill might not necessarily align with intended course learning outcomes.

4.3 Thinking time helped students compose answers, but also led some students to feel surprised when asked follow-up questions

The 10 minutes of thinking time introduced at the beginning of each section for all students (section 3.4.1) was intended to help students compose their responses. Since students were also encouraged to take notes during the thinking time and use those notes to support their code interview, student preparation strategies could have a significant influence on code interview performance.

In particular, some TAs observed that live elements of the code interview caught students off-guard. Following prior work [3], TAs were directed to ask follow-up questions to better assess student proficiency. Since students had just prepared for the code interview and brought with them their notes, they may have believed that those notes would have prepared them for the entire activity. Furthermore, the specific follow-up questions were prepared by the teaching team in advance, so when asked in the context of the live interview, they might not have reflected the state of the conversation or matched the student's reasoning at that time.

4.4 Waiting time variability raises new equity and fairness issues as well as a trade-off between equality and equity in assessment

Positioning thinking time as an activity that occurs at the beginning of section meant that the time that students had to wait for a TA to reach them varied drastically (section 3.4.1). The last student to conduct an interview may have had to wait up to 35 minutes after finishing their thinking time before conducting their code interview. Considering that student technology use was prohibited during this waiting time, the anticipation of waiting for your turn could add to test-induced stress or anxiety. Some students may prefer to interview as soon as possible, while others may prefer to interview as late as possible, and these preferences could pose new equity and fairness issues. Although the use of group discussions and exit tickets as preparation for the code interview served as a way to mitigate some of the negative effects of this design change, there may yet be differences in student performance and self-reported stress level between students who interview as soon as possible versus as late as possible.

For courses that wish to replicate this code interview format within scheduled class time, we recommend addressing waiting time by implementing a sign-up system. This sign-up system could provide affordances for students to receive interview slots that better align with their assessment preferences. For example, a student who prefers to address test anxiety by getting it done as soon as

possible can do so, while a student who prefers to address test anxiety by taking more time to think can also do so. But if waiting time is also a form of thinking time for some students, is it fair for some students to receive different amounts of waiting time than others? This tension raises a new question about the trade-offs between equality and equity in preparing students for code interviews.

5 CONCLUSION

Our proposed code interview protocol offers a more authentic assessment that evaluates both student proficiency with course learning outcomes and their communication skills. Our approach addresses some of the challenges in administering code interviews identified by prior work through 4 key structures:

- Focusing code interviews on the most important assignments.
- Practicing oral communication with collaborative peer discussions.
- Priming content knowledge with pre-interview exit tickets.
- Restructuring code interviews to reduce stress.

While this experience report does not present conclusive results, the preliminary reflections raise new concerns for future work on code interviews as an assessment methodology:

- Students self-identifying as English language learners may face unique difficulties if oral assessment triggers new anxieties when they have otherwise felt confident in English.
- Tensions between equality and equity in designing components of the code interview such as waiting time, thinking time, and swapping-out assigned questions.
- Avoiding unintended hints: item design for questions involving specific content knowledge that may unintentionally overlap with other questions in the question bank.

Ultimately, this experience report contributes to the field:

(1) A code interview protocol addressing difficulties identified in prior work around preparation and stressfulness.
(2) Preliminary reflections on the protocol by the teaching team, highlighting nuances in the protocol implementation.
(3) Implications for future work including raising new questions, tensions, and concerns.

In addition to the questions about code interview equity, preparation, and stressfulness raised by this work, future work should also investigate more nuanced considerations around accessibility and the experiences of students with disabilities of all kinds. How might their perspectives inform the design of more equitable code interviews? How does the specific contexts where learning takes place shape students' experiences with code interviews? And what kinds of training can we provide to TAs (and instructors) to help them realize the potential of code interviews as a way to learn and develop their communication skills?

REFERENCES

[1] Giulia Alberini and Elena Bai. 2025. Implementation of Technical Interviews as an Alternative Assessment in a Large Introductory CS Course. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2* (Pittsburgh, PA, USA) *(SIGCSE TS 2025)*. Association for Computing Machinery, New York, NY, USA, 1361–1362. https://doi.org/10.1145/3641555.3705200

[2] Emeka Anaza, Paul Mabrey, Mikihiro Sato, Olivia Miller, and Julia Thompson. 2023. Improving Student Interview Preparation Through Collaborative Multimodal Mock-Interview Assignments. *Sport Management Education Journal* 17, 2 (2023), 164 – 176. https://doi.org/10.1123/smej.2021-0021

[3] Suhas Kannam, Yuri Yang, Aarya Dharm, and Kevin Lin. 2025. Code Interviews: Design and Evaluation of a More Authentic Assessment for Introductory Programming Assignments. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) *(SIGCSETS 2025)*. Association for Computing Machinery, New York, NY, USA, 554–560. https://doi.org/10.1145/3641554.3701806

[4] Terence Kelly. 2024. Programmer Job Interviews: The Hidden Agenda. *Queue* 21, 6 (Jan. 2024), 16–26. https://doi.org/10.1145/3639444

[5] Clara S Kim, Ching-Hsiu Chiu, and Tobias K Boehm. 2024. Developing students' collaboration and critical thinking skills via oral examination. *J. Dent. Educ.* 88 Suppl 3 (Dec. 2024), 1892–1894.

[6] Peter Ohmann. 2019. An Assessment of Oral Exams in Introductory CS. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) *(SIGCSE '19)*. Association for Computing Machinery, New York, NY, USA, 613–619. https://doi.org/10.1145/3287324.3287489

[7] Scott J. Reckinger and Shanon M. Reckinger. 2022. A Study of the Effects of Oral Proficiency Exams in Introductory Programming Courses on Underrepresented Groups. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1* (Providence, RI, USA) *(SIGCSE 2022)*. Association for Computing Machinery, New York, NY, USA, 633–639. https://doi.org/10.1145/3478431.3499382

[8] Allison S. Theobold. 2021. Oral Exams: A More Meaningful Assessment of Students' Understanding. *Journal of Statistics and Data Science Education* 29, 2 (2021), 156–159. https://doi.org/10.1080/26939169.2021.1914527 arXiv:https://doi.org/10.1080/26939169.2021.1914527